Order and Payment API

Implementation and API guide

This Laravel 12 and PHP 8.3 project manages owned orders and simulated payments through a service layer. This guide explains how to run it, call every endpoint, follow the payment rules, and add another gateway. The supplied credit card and PayPal drivers simulate results and never charge money.

Start the project

Install PHP 8.3 or newer, Composer 2, Laravel requirements and pdo_sqlite. No Node.js or external payment account is required. Run these commands inside the extracted project folder.

```
composer install
php -r "file_exists('.env') || copy('.env.example', '.env');"
php artisan key:generate
php artisan jwt:secret
php -r "file_exists('database/database.sqlite') || touch('database/database.sqlite');"
php artisan migrate
php artisan serve
```

Base URL: http://127.0.0.1:8000/api/v1
Health check: GET /up. Register an account through the API; no default account is seeded.

Use the Postman collection

Import Order_Payment_API.postman_collection.json and run its four folders in order. It includes all variables. The optional Local.postman_environment.json overrides base_url only. Registration saves the JWT, order creation saves IDs, and payment requests generate or reuse UUID keys automatically.

Folder	What it demonstrates
1 Authentication	Registration, login and current account
2 Orders	Create, update, confirm, show, list and delete
3 Payments	Decline, retry, replay, listings and PayPal
4 Business Rule Examples	Expected 401, 404, 409 and 422 responses

Authentication

Send JSON request bodies and Accept: application/json. All endpoints except registration and login require Authorization: Bearer <access_token>. API authentication uses JWT rather than a browser session.

Method	Path	Success
POST	/auth/register	201
POST	/auth/login	200
GET	/auth/me	200

Register an account

```
{
  "name": "Mohamed Sayed",
  "email": "mohamed@example.com",
  "password": "SecurePass123",
  "password_confirmation": "SecurePass123"
}
```

Emails are trimmed and lowercased. Names are limited to 120 characters and emails to 255. Passwords need at least 10 characters, both letter cases and a number. The maximum is 72 bytes, with no null bytes, to avoid bcrypt truncation. Confirmation must match.

Log in

```
{
  "email": "mohamed@example.com",
  "password": "SecurePass123"
}
```

Token response

```
{
  "data": {
    "access_token": "<signed-jwt>",
    "token_type": "Bearer",
    "expires_in": 3600,
    "user": {
      "id": 1, "name": "Mohamed Sayed",
      "email": "mohamed@example.com"
    }
  }
}
```

GET /auth/me returns the user object inside data. Invalid credentials return 401 with the message Invalid email or password. Tokens expire after 60 minutes by default. JWT_SECRET is generated separately from APP_KEY; the API verifies HS256 signatures, issuer, audience, subject and time claims. Refresh tokens and individual-token logout are outside this task.

Create and update orders

Every order belongs to the authenticated account. Customer information is a name and email snapshot supplied with the order. Another account cannot read or modify that order.

Method	Path	Success
POST	/orders	201
GET	/orders/{id}	200
PATCH / PUT	/orders/{id}	200
DELETE	/orders/{id}	204

Create order request

```
{
  "customer": {
    "name": "Customer",
    "email": "customer@example.com"
  },
  "status": "pending",
  "items": [
    {"product_name": "Keyboard", "quantity": 2, "price": "19.99"},
    {"product_name": "Cable", "quantity": 3, "price": "0.10"}
  ]
}
```

The server calculates 2 x 19.99 + 3 x 0.10 = 40.28 USD. The database stores integer cents; response amounts are decimal strings. Do not send total, total_minor, user_id or currency.

Input limits

Field	Rule
customer	Both name and email required; name max 120
items	1 to 100 items
product_name	Required string, maximum 200 characters
quantity	Integer from 1 to 10000
price	Positive, up to 999999.99, at most 2 decimal places
status	Defaults to pending; confirmed allowed on creation

Update and delete

```
PATCH /orders/1
{"status": "confirmed"}
```

PATCH and PUT update supplied fields. Supplying items replaces the entire item list and recalculates the total. Supplying customer requires both name and email. An order may move from pending to confirmed or cancelled, or from confirmed to cancelled. Cancelled orders cannot be edited. Deletion returns an empty 204 response only when no payment attempts exist.

Order responses and pagination

Create, show and update return the same order shape. Identifiers and timestamps below are examples. List endpoints place these objects directly in data, without a second data wrapper.

```
{
  "data": {
    "id": 1,
    "customer": {"name": "Customer", "email": "customer@example.com"},
    "status": "confirmed",
    "currency": "USD",
    "total": "40.28",
    "items": [
      {
        "id": 1, "product_name": "Keyboard", "quantity": 2,
        "price": "19.99", "line_total": "39.98"
      },
      {
        "id": 2, "product_name": "Cable", "quantity": 3,
        "price": "0.10", "line_total": "0.30"
      }
    ],
    "created_at": "2026-10-19T23:00:00.000000Z",
    "updated_at": "2026-10-19T23:00:00.000000Z"
  }
}
```

List the current account orders

```
GET /orders?status=confirmed&per_page=15&page=1
```

Parameter	Accepted values
status	Optional: pending, confirmed or cancelled
per_page	Optional integer from 1 to 100; default 15
page	Optional positive integer; default 1

Results sort by newest order ID. The service eager-loads items and preserves validated filters in page links. The response has data, links and meta. The shared envelope is shown below in abbreviated form; the API also supplies Laravel numbered-page links and range values.

```
{
  "data": ["<order objects>"],
  "links": {"first": "<url>", "last": "<url>", "prev": null, "next": null},
  "meta": {"current_page": 1, "per_page": 15, "total": 1, "last_page": 1}
}
```

A missing or foreign-owned order returns 404. Invalid filters return 422 with field-specific errors. Follow links for navigation rather than parsing their labels.

Process a payment

POST /orders/{id}/payments pays the full stored order total. The order must be confirmed and have no pending or successful payment. Choose credit_card or paypal. Do not send an arbitrary amount, card number or CVV.

Required headers and body

```
Authorization: Bearer <access_token>
Idempotency-Key: 123e4567-e89b-42d3-a456-426614174000
Content-Type: application/json

{"method": "credit_card", "simulation_result": "successful"}
```

The Idempotency-Key must be a UUID. simulation_result defaults to successful; failed simulates a decline. A new attempt returns 201 and Idempotent-Replayed: false. A decline also creates a resource and returns 201, with status failed.

Payment response

```
{
  "data": {
    "id": "019a0d00-0000-7000-8000-000000000001",
    "order_id": 1,
    "status": "successful",
    "method": "credit_card",
    "amount": "40.28",
    "currency": "USD",
    "gateway_reference": "sim_cc_<payment-uuid>",
    "failure_code": null,
    "created_at": "2026-10-19T23:00:00.000000Z",
    "updated_at": "2026-10-19T23:00:00.000000Z"
  }
}
```

Replay and retry rules

Situation	Result
Same key and same request	200, same payment; Idempotent-Replayed: true
Same key with changed order, method or outcome	409 idempotency_key_reused
Failed attempt and a new key	A new payment attempt is allowed
Pending or successful attempt and a new key	409 order_already_paid_or_processing

Keys are scoped per account and have no expiry. Omitted successful and explicit successful are equivalent. Decline codes are card_declined and paypal_declined. PayPal references start with sim_paypal_; credit card references start with sim_cc_. A successful payment leaves the order status confirmed.

Read payments and handle errors

Method	Path	Response
GET	/payments/{uuid}	One owned payment
GET	/payments	Paginated owned payments
GET	/orders/{id}/payments	Paginated attempts for an owned order

Both payment lists accept page, per_page and optional status: pending, successful or failed. They use the same pagination envelope as orders and sort by created_at descending, then payment UUID descending. Any order filter is applied after verifying ownership. Keys and request fingerprints are not exposed.

HTTP errors

Status	Meaning
401	Missing, invalid or expired JWT, or incorrect credentials
404	Missing record or record owned by another account
409	A business rule prevents the operation
422	Invalid request data, filter, method or idempotency header
429	Rate limit exceeded; inspect Retry-After
500	Unexpected error; details are hidden when APP_DEBUG=false

Business conflict example

```
{
  "message": "Payments can only be processed for confirmed orders.",
  "error_code": "order_not_confirmed"
}
```

Validation error example

```
{
  "message": "The idempotency key field is required.",
  "errors": {
    "idempotency_key": ["The idempotency key field is required."]
  }
}
```

Other conflict codes are invalid_order_status, invalid_status_transition, order_cancelled, order_has_payments, order_payment_active, order_already_paid_or_processing and idempotency_key_reused. Clients should branch on status and error_code or validation field keys.

Any payment attempt prevents deleting the order or changing its customer/items. Pending or successful attempts prevent cancellation. Failed-only orders may be cancelled or retried. Authentication allows 10 requests per minute per IP and 5 per email plus IP; protected routes allow 120 per minute per account.

Service structure and readable code

Controllers accept validated input, call a service and choose the HTTP response. All account, order and payment database queries live in services. FormRequests validate input and Resources format output.

Class	Responsibility
AuthService	Register accounts and verify login credentials
JwtService	Issue tokens and verify their signature and claims
OrderService	Owned queries, totals, transitions and transactional writes
PaymentService	Owned queries, idempotency and payment processing
GatewayRegistry	Select the configured enabled payment driver
PaymentGateway	Contract implemented by each payment strategy
OrderItemResource	Format each line item without a nested mapping closure

PaymentService processing flow

The public process method reads in this order: lock the owner and order, build the request fingerprint, replay an existing matching payment, check payment rules, create pending, run the strategy, save the result. The private method names describe these operations directly.

```
$order = $this->lockOrderForPayment($user, $orderId);
$requestHash = $this->requestFingerprint($order, $method, $simulationResult);
$existing = $this->findExistingPayment($user, $key, $requestHash);

if ($existing !== null) {
    return new ProcessedPayment($existing, replayed: true);
}

$this->ensureOrderCanBePaid($order);
$payment = $this->createPendingPayment($user, $order, $data, $requestHash);
$gatewayResult = $this->chargePayment($payment, $simulationResult);
```

This excerpt shows the service sequence; the complete method also normalizes the simulator default, stores the gateway result and wraps the work in a transaction. OrderService follows the same approach with ensureOrderCanBeUpdated, changeStatus and replaceItems.

Transaction boundaries

Writes lock the owned order. Payments lock the account first, then the order, so concurrent requests reuse keys consistently. Foreign keys restrict deletion and a unique account/key index backs idempotency. SQLite is suitable for the local demonstration; MySQL is needed for row-lock concurrency. The request fingerprint format remains compatible with existing payment keys.

Add a gateway and verify changes

Create one PaymentGateway implementation and register it in config/payments.php. The registry and request validation discover the new method; controllers and PaymentService need no switch statement or method-specific edit.

```
final class BankTransferGateway implements PaymentGateway
{
    public function charge(PaymentContext $context): GatewayResult
    {
        $failed = $context->simulationResult === 'failed';

        return new GatewayResult(
            $failed ? PaymentStatus::Failed : PaymentStatus::Successful,
            'sim_bank_'.$context->paymentId,
            $failed ? 'bank_declined' : null,
        );
    }
}
```

Use the App\Payments\Gateways namespace and import PaymentGateway, PaymentContext, GatewayResult and PaymentStatus. A complete copyable example with imports is in docs/GATEWAYS.md.

```
// Add under gateways in config/payments.php:
'bank_transfer' => [
    'driver' => BankTransferGateway::class,
    'enabled' => (bool) env('BANK_TRANSFER_ENABLED', true),
    'api_key' => env('BANK_TRANSFER_API_KEY'),
],
```

Import the driver class in that configuration file, put credentials in .env, and run php artisan config:clear. Send method bank_transfer with a fresh key for a fresh confirmed order.

Verify the project

```
composer test
composer lint
composer validate --strict
composer audit
```

The regression suite covers ownership, exact totals, status transitions, both gateways, retries, idempotency and rollback. The Postman runner exercises the complete HTTP workflow, including intentional errors. The default suite uses isolated in-memory SQLite. GitHub CI also contains PHP 8.4 and MySQL jobs; local verification does not establish their results or production load behavior.

Connecting a real provider

The supplied drivers are local simulators. A real provider needs a durable pending attempt and outbox, provider idempotency, authenticated deduplicated webhooks, and reconciliation for unknown outcomes. Network calls must run outside the simulator transaction. A SQL rollback cannot reverse an external charge. See docs/ARCHITECTURE.md for that boundary.